PERTANDINGAN WORLDSKILLS MALAYSIA KATEGORI PENGAJAR (WSMP) TAHUN 2025

(WEB TECHNOLOGIES)

SOALAN PRAKTIKAL

PERINGKAT AKHIR

MASA : 17 JAM

NAMA: _____

No. KP: _____

JABATAN PEMBANGUNAN KEMAHIRAN
KEMENTERIAN SUMBER MANUSIA

MODULE A (4 HOURS)

CONTENTS

This module has the following files:
Module_A_Media_Files.zip

INTRODUCTION

In this speed Test Project. You only need to complete the randomly picked one during the competition. Please follow the instructions to know which one to implement.

For each Easy, it is worth 0.5 point.
For each Medium (Normal), it is worth 1.0 point.
For each Difficult (Hard), it is worth 1.5 points.
For each Very Difficult (Very Hard), it is worth 2 points.

Please put your files **in /XX_module_a/** and with the **task ID** as folder name, e.g. **/XX_module_a/A1/**
The **XX** is the workstation number.

DESIGN IMPLEMENTATION

SET 1

A1: 4-corners image (Easy)

Create an image gallery where hovering over a placeholder reveals the full image with a smooth transition effect.

Requirements:

- 4 image placeholders in a 2x2 grid
- Initially show greyscale/blurred version
- On hover: transform to full color with smooth transition
- Transition duration: 0.5 seconds
- Add a subtle zoom effect (1.1x scale) on hover
- Grid gap: 20px
- Center the entire grid on the page

Deliverables:

- 4 images with hover reveal effect
- Smooth color and scale transitions
- Properly centered grid layout

A2: Rotating Cube (Normal)

Create a 3D rotating cube using CSS transforms that continuously rotates on multiple axes.

Requirements:

- Cube dimensions: 150px × 150px × 150px
- Each face has a different color:
- Front: #3498db (blue)
- Back: #e74c3c (red)
- Right: #2ecc71 (green)
- Left: #f39c12 (orange)
- Top: #9b59b6 (purple)
- Bottom: #1abc9c (turquoise)
- Continuous rotation on both X and Y axes
- Full rotation: 10 seconds
- Cube centered on viewport
- Apply perspective for 3D effect
- Use CSS animations only (no JavaScript)

Media Files:

Example video showing rotation behavior

A3: Animated Progress Circle in Pure CSS (Difficult)

Create an animated circular progress indicator that fills from 0% to 100% using only CSS.

Requirements:

- Use HTML and CSS only (no JavaScript, no SVG)
- Circle diameter: 200px
- Progress color: #27ae60 (green)
- Background track color: #ecf0f1 (light grey)
- Border width: 15px
- Animation fills from 0% to 75% over 2 seconds
- Display percentage text in center (75%)
- Centered horizontally and vertically on page
- Given: index.html; style.css
- Modify ONLY the style.css file

Summary:

- Circular progress indicator created with CSS
- Smooth animation from 0% to 75%
- Percentage text displayed in center
- Proper centering on all screen sizes
- No JavaScript or SVG used

A4: Pure CSS Menu with Mega Dropdown (Very Difficult)

Create a navigation menu with expandable mega dropdown

Requirements:

- Navigation menu with expandable mega dropdown.
- 4 menu items, hover on item 2 shows 3-column dropdown (300px height).
- Columns have headers + 5 links each.
- Smooth slide-down animation (0.4s).
- Dropdown closes when mouse leaves.
- Menu width 100%, items centered.
- Active item: #3498db background.
- Dropdown background: white with shadow.
- CSS only (no JS).
- *Files: index.html, style.css (modify CSS only)*

SET 2

A1: Card Flip on Click (Easy)

Create a clickable card that flips to reveal information on the back using CSS checkbox technique.

Requirements:

- Card size: 280px × 380px
- Front: Display an image with title
- Back: Display text content - Click to flip (no hover)
- 3D flip animation (0.8 seconds)
- Card centered on page
- Use CSS checkbox hack for click functionality

Deliverables:

- Front and back card faces
- Click-to-flip functionality
- Smooth 3D flip animation

A2: Animated Wave Effect (Normal)

Create an animated wave pattern using CSS that moves continuously across the screen.

Requirements:

- Three overlapping wave layers
- Wave colors (from back to front):
  - Layer 1: rgba(52, 152, 219, 0.4)
  - Layer 2: rgba(52, 152, 219, 0.6)
  - Layer 3: rgba(52, 152, 219, 0.8)
- Waves animate from right to left
- Different animation speeds for depth effect:
  - Layer 1: 15 seconds
  - Layer 2: 10 seconds
  - Layer 3: 7 seconds
- Waves at bottom of viewport
- Height: 150px
- Smooth, continuous looping animation

A3: Yin-Yang Symbol in Pure CSS (Difficult)

Create the yin-yang symbol using only HTML and CSS.

Requirements:

- Symbol diameter: 300px
- Use only CSS (no images, no SVG)
- Proper yin-yang proportions:
- Black and white halves
- Two smaller circles (one black, one white)
- Two small dots (opposite colors)
- Centered horizontally and vertically
- Perfect circular shapes
- Colors: pure black (#000) and pure white (#fff)
- Given: index.html; style.css
- Modify ONLY the style.css file

Summary:

- Accurate yin-yang symbol
- Correct proportions and positioning
- Pure CSS implementation
- Centered on viewport

A4: Animated Gradient Background (Very Difficult)

Create a full-screen animated gradient background shifting between 4 colors.

Requirements:

- Colors cycle: #667eea → #764ba2 → #f093fb → #4facfe → repeat.
- Complete cycle: 12s.
- Smooth color transitions.
- Add floating geometric shapes (3 circles, different sizes: 100px/150px/200px) that move slowly across screen with different speeds/directions.
- Shapes semi-transparent (opacity 0.2).
- CSS only. *Files: index.html, style.css (modify CSS only)*

SET 3

A1: Image Slider with CSS (Easy)

Create a simple image slider with previous/next buttons using CSS checkbox technique.

Requirements:

- 3 images in sequence
- Previous and Next buttons
- Clicking buttons changes visible image
- Smooth slide transition (0.5 seconds)
- Container size: 600px × 400px
- Centered on page
- Use CSS checkbox hack for navigation

Deliverables:

- 3-image slider
- Working prev/next navigation
- Smooth transitions

A2: Pulsing Ripple Effect (Normal)

Create expanding ripple circles that pulse outward continuously from a central point.

Requirements:

- Central circle diameter: 80px
- Central circle color: #3498db (blue)
- Three ripple waves expanding outward
- Each ripple:
    - Starts at center size
    - Expands to 250px diameter
    - Fades out during expansion
- Duration: 3 seconds
- Ripples stagger (0.5 second delay between each)
- Infinite animation
- Centered on viewport
- CSS animations only

A3: CSS-Only Clock Face (Difficult)

Create a clock face with hour markers using only CSS.

Requirements:

- Clock diameter: 400px
- 12 hour markers positioned correctly
- Markers at 12, 3, 6, 9 are larger/different style
- Clock border: 4px solid black

- Markers created using CSS (no images)
- Center dot for clock hands mounting point
- All markers properly rotated and positioned
- Clock face background: white (#fff)
- Centered horizontally and vertically
- Given: index.html; style.css
- Modify ONLY the style.css file

Summary:

- 12 hour markers in correct positions
- Different styles for cardinal positions
- Accurate rotation for each marker
- Professional clock face appearance

A4: CSS-Only Image Comparison Slider (Very Difficult)

Create a before/after image comparison slider.

Requirements:

- 2 images overlapped (600×400px).
- Draggable divider line in middle (vertical).
- Left side: “before” image, right side: “after” image.
- Divider has handle (circle, 40px).
- Drag divider left/right to reveal more/less of each image.
- Use CSS resize property + range input trick.
- Centered on page.
- *Files: index.html, style.css (modify CSS only)*

FRONT-END DEVELOPMENT

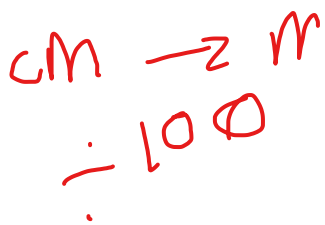
SET 1

B1: BMI Calculator (Easy)

Create a simple BMI (Body Mass Index) calculator with instant results.

Requirements:

- Input fields for:
  - Weight (kg)
  - Height (cm)
- Calculate and display BMI on button click
- Show BMI category:
  - Underweight: < 18.5
  - Normal: 18.5 - 24.9
  - Overweight: 25 - 29.9
  - Obese: ≥ 30
- Color-code the result:
  - Underweight: #3498db (blue)
  - Normal: #2ecc71 (green)
  - Overweight: #f39c12 (orange)
  - Obese: #e74c3c (red)
- Input validation (numbers only, positive values)
- Formula: BMI = weight(kg) / (height(m))²
- Use vanilla JavaScript only



B2: String Pattern Matching with Wildcards (Normal)

Implement a string pattern matching algorithm that supports wildcard characters.

Requirements:

- Function matchPattern(text, pattern) returns true/false
- Support two wildcards:
  - * matches zero or more characters
  - ? matches exactly one character
- Case-sensitive matching
- Examples:
  - matchPattern("hello", "h*o") → true
  - matchPattern("hello", "h?llo") → true
  - matchPattern("hello", "h?l") → false
  - matchPattern("test", "*est") → true
  - matchPattern("test", "t*t") → true
  - matchPattern("hello", "h*l*o") → true
- No external libraries
- Implement using recursion or dynamic programming
- Display test results on webpage

Constraints:

- Text length: 1-100 characters
- Pattern length: 1-100 characters
- Only alphanumeric characters plus wildcards

B3: Analog Clock (Difficult)

Create a working analog clock that displays the current time with moving hands.

Requirements:

- Clock face with hour markers (1-12)
- Three hands:
  - Hour hand (short, thick)
  - Minute hand (medium length)
  - Second hand (long, thin, red)
- Hands move smoothly to current time
- Update every second
- Clock diameter: 300px
- Center point/pivot for hands
- Hour markers at correct positions
- Smooth continuous movement (not jerky steps)
- Use JavaScript to calculate hand positions
- Accurate time display

Visual requirements:

- Hour hand: #333, width: 6px, length: 80px
- Minute hand: #555, width: 4px, length: 110px
- Second hand: #e74c3c, width: 2px, length: 120px
- Clock face: white with black border
- Hour markers: small circles or lines

B4: Memory Card Matching Game (Very Difficult)

Create a memory card game with 4×4 grid (16 cards), 8 matching pairs.

Requirements:

- Click card to flip, show hidden image/icon.
- Find matching pairs.
- Match: cards stay revealed (green border).
- No match: flip back after 1s.
- Track: moves count, time elapsed, pairs found.
- Win message when all matched.
- “Reset” button.
- Card flip animation (3D, 0.6s).
- Shuffle cards on start/reset.
- LocalStorage: save best score (fewest moves).
- Vanilla JS

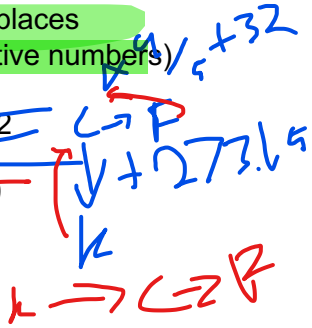
SET 2

B1: Temperature Unit Converter (Easy)

Create a temperature converter between Celsius, Fahrenheit, and Kelvin.

Requirements:

- Three input fields (Celsius, Fahrenheit, Kelvin)
- Typing in any field updates the other two automatically
- Real-time conversion (no submit button)
- Round results to 2 decimal places
- Input validation (allow negative numbers)
- Formulas:
  - C to F: (C × 9/5) + 32
  - C to K: C + 273.15
  - F to C: (F - 32) × 5/9
  - K to C: K - 273.15
- Use vanilla JavaScript only



B2: Image Slider with Thumbnail Navigation (Normal)

Create an interactive image slider with thumbnail navigation.

Requirements:

Main Display:

- Main image display area (800px × 500px)
- 5 images in the slider

Thumbnail Navigation:

- Show all 5 thumbnails below main image (150px × 100px each)
- Active thumbnail has colored border
- Click thumbnail to jump to that image

Navigation Buttons:

- Previous button (◄)
- Next button (►)
- Image counter (e.g., "3 / 5")

Features:

- Smooth fade transition between images (0.5s)
- Disable "Previous" button on first image
- Disable "Next" button on last image

Use vanilla JavaScript only

B3: Interactive Chart Builder (Difficult)

Create an interactive chart builder that generates bar charts from user data.

Requirements:

- Input form for chart data:
- Chart title
- Add/remove data points (label and value)
- Minimum 3 data points, maximum 10
- Generate animated bar chart:
    - Bars grow from 0 to full height
    - Animation duration: 1 second
    - Different color for each bar
- Chart features:
    - Y-axis with scale labels
    - X-axis with data labels
    - Value display on hover over bars
    - Responsive width (fits container)
- Chart styling:
    - Bar colors from color palette
    - Grid lines on Y-axis
    - Clean, professional appearance
- Export chart as image (PNG)
- Use vanilla JavaScript and Canvas API
- No charting libraries

B4: Expense Tracker Application (Very Difficult)

Create an expense tracking application.

Requirements:

- Add expenses: description, amount, category (Food/Transport/Shopping/Bills/Other), date.
- Display list: all expenses sorted by date (newest first).
- Show totals: by category (pie chart visual or bars) + grand total.
- Filter: by category, by date range.
- Delete expense.
- Edit expense (inline editing).
- Data in LocalStorage, restore on reload.
- Export to CSV.
- Monthly summary: total spent + breakdown by category.
- Vanilla JS.

SET 3

B1: Tip Calculator (Easy)

Create a tip calculator for restaurant bills.

Requirements:

- Input fields:
  - Bill amount
  - Tip percentage (15%, 18%, 20%, custom)
  - Number of people
- Calculate and display:
  - Tip amount
  - Total bill (bill + tip)
  - Amount per person
  - Update in real-time
  - Input validation (positive numbers only)
  - Default tip: 15%
  - Format currency to 2 decimal places
- Use vanilla JavaScript only

B2: Filterable Product Grid (Normal)

Create a product grid with filtering functionality.

Requirements:

- Display 12 products in a responsive grid:
  - 3 columns on desktop
  - 2 columns on tablet
  - 1 column on mobile
- Each product has:
  - Name
  - Category (Electronics, Clothing, Books, Sports)
  - Price
  - Image (placeholder images OK)

Product Card Display:

- Product image
- Product name
- Category label
- Price (formatted with RM)
- "Add to Cart" button (non-functional)

Filtering:

- Category filter (dropdown): All, Electronics, Clothing, Books, Sports
- Search box: Filter by product name (case-insensitive)
- "Clear Filters" button to reset all filters

Features:

- Show product count (e.g., "Showing 8 of 12 products")
- Display "No products found" when filters return no results
- Smooth fade animation when products appear/disappear
- Search updates as you type (300ms debounce)
- Use vanilla JavaScript only

B3: Drag and Drop Kanban Board (Difficult)

Create a Kanban board with drag-and-drop task management.

Requirements:

- Three columns: "To Do", "In Progress", "Done"
- Add new tasks (input field + button in each column)
- Drag tasks between columns using HTML5 Drag and Drop API
- Delete tasks (delete button on each task)
- Each task displays:
  - Task title
  - Delete button
- Task counter showing number of tasks per column

Drag and Drop:

- Dragged task has reduced opacity (0.5)
- Drop zone shows visual highlight
- Smooth animations when moving tasks

Data Persistence:

- Save all tasks to localStorage
- Restore board state when page reloads

Use vanilla JavaScript only (no frameworks)

B4: Weather Dashboard Application (Very Difficult)

Create a weather dashboard displaying weather for 3 cities (provided in JSON).

Requirements:

- Show per city: name, temperature (°C), condition (sunny/cloudy/rainy), humidity %, wind speed.
- Weather icons change based on condition.
- Toggle °C/°F.
- Search: add new city (filter from 20+ city list).
- Current city highlighted.
- Auto-refresh data every 30s (simulate with random temp ±2°).
- Sort cities: by name (A-Z), by temperature.
- LocalStorage: save city preferences.
- Vanilla JS.
- *Data: cities.json with weather info provided*

BACK-END DEVELOPMENT

SET 1

C1: Vowel and Consonant Counter (Easy)

Create a server-side form that analyzes text and counts vowels and consonants.

Requirements:

- Textarea input for text
- Submit button
- Server processes and displays:
  - Total vowels (a, e, i, o, u - case insensitive)
  - Total consonants (all other letters)
  - Total letters (excluding spaces and punctuation)

Example:

- Input: "Hello World!"
- Output:
  - Total vowels: 3
  - Total consonants: 7
  - Total letters: 10

Files: XX_C1.php or XX_C1.js

C2: Color Palette Generator (Normal)

Create a server-side application that generates a color palette.

Requirements:

- Input: One base color (color picker or hex code input)
- Generate 5 colors:
  - Original color
  - Lighter version (+30% brightness)
  - Darker version (-30% brightness)
  - Complementary color (opposite on color wheel)
  - Random accent color
- Display each color:
  - Color swatch (80px × 80px)
  - Hex code
  - RGB values

Layout:

- Display 5 color swatches in a row
- Show hex and RGB values below each swatch

Files: XX_C2.php or XX_C2.js

C3: Image Gallery with Upload (Difficult)

Create a server-side image gallery with upload functionality.

Upload Features:

- Multiple file upload (select multiple files at once)
- File validation:
  - Only accept: jpg, png, gif
  - Maximum size: 2MB per file
- Display upload progress indicator

Image Processing:

- Generate thumbnail (200×200px) for each uploaded image
- Store original image
- Maintain aspect ratio when creating thumbnails

Gallery Display:

- Grid of thumbnail images
- Click thumbnail to view full-size image
- Display image filename and upload date

Delete Function:

- Delete button on each image
- Remove file from server when deleted

File Storage:

- Folder structure:
  - /uploads/original/
  - /uploads/thumbnails/

Technical Requirements:

- Server-side image processing (GD, ImageMagick)
- File type validation (check extension and MIME type)
- Store image metadata in JSON file or database

Files: XX_C3.php or XX_C3.js (+ database schema if used)

C4: Poll/Voting System (Very Difficult)

Create a polling system. Create poll (question + 4 options).

Requirements:

- Vote once per session/IP.
- View results: option text, vote count, percentage bar, total votes.
- Results page: bar chart showing all options with %.
- Multiple polls support: list all polls, click to vote/view results.
- Admin: create poll, delete poll, view all votes.
- Prevent duplicate voting (session + IP check).
- DB schema: polls(id,question,created_at), options(id,poll_id,option_text,votes), votes(id,poll_id,option_id,ip_address,voted_at).
- *Files: XX_C4.php or XX_C4.js + schema*

SET 2

C1: Word Frequency Counter (Easy)

Create a server-side application that counts word frequency in text.

Requirements:

- Textarea input for text
- Submit button
- Server processes and displays:
  - Total word count
  - List of unique words with their frequency
  - Sort by frequency (highest to lowest)
  - Show top 10 most common words

Example:

- Input: "hello world hello everyone welcome to the world"
- Output:
  - Total words: 8
  - Most frequent words:
    - hello: 2
    - world: 2
    - everyone: 1
    - welcome: 1
    - to: 1
    - the: 1

Files: XX_C1.php or XX_C1.js

C2: PDF Report Generator (Normal) - 1.0 point

Create a server-side PDF report generator.

Requirements:

- Input form with fields:
  - Report title
  - Author name
  - Report content (textarea)
- Generate PDF with:
  - Title at top
  - Author name
  - Content (formatted paragraphs)
  - Page numbers
- Download generated PDF

Use server-side PDF library:

- PHP: FPDF or TCPDF
- Node.js: PDFKit

Files: XX_C2.php or XX_C2.js

C3: Contact Form with Email and Database (Difficult)

Create a contact form that sends emails and stores submissions.

Requirements:

Contact Form Fields:

- Name (required)
- Email (required)
- Subject (required)
- Message (required, min 10 characters)

Server-Side Validation:

- Check all required fields are filled
- Validate email format
- Check message minimum length

Email Functionality:

- Send email to site admin with form data
- Display success message after submission

Database Storage:

- Store all submissions with timestamp
- Track if email was sent successfully

Admin Panel:

- View all submissions in a list
- Display: name, email, subject, date
- Delete submissions

Security:

- Prevent SQL injection (use prepared statements)
- Sanitize user input (prevent XSS)

Database Schema:

- contact_submissions:
- id, name, email, subject, message
- submitted_at, email_sent

Files: XX_C3.php or XX_C3.js (+ database schema)

C4: URL Shortener Service (Very Difficult)

Create a URL shortening service.

Requirements:

- Shorten URLs: input long URL → generate short code (6 random chars).
- Store: original URL, short code, created date, click count.
- Redirect: visit /s/ABC123 → redirect to original URL + increment counter.
- List all URLs: show original (truncated), short link, clicks, date.
- Search URLs by original URL.
- Delete URL.
- Analytics: total URLs created, total clicks, most clicked URL.
- Validate URL format.
- Prevent duplicate short codes.
- DB schema: urls(id,original_url,short_code,clicks,created_at).
- *Files: XX_C4.php or XX_C4.js + schema*

SET 3

C1: Simple Calculator (Easy)

Create a server-side calculator for basic operations.

Requirements:

- Input fields:
  - First number
  - Second number
  - Operation selector (dropdown: Add, Subtract, Multiply, Divide)
- Submit button
- Server processes and displays:
  - The calculation result
  - Show the equation (e.g., "5 + 3 = 8")

Validation:

- Check inputs are valid numbers
- Handle division by zero (show error message)

Example:

- Input: 10, 5, Divide
- Output: "10 ÷ 5 = 2"

Files: XX_C1.php or XX_C1.js

C2: CSV to HTML Table Converter (Normal)

Create a server-side CSV to HTML table converter.

Requirements:

- Upload CSV file
- Parse CSV content
- Display data as HTML table

Features:

- First row treated as table headers
- Display all rows in table format
- Basic table styling:
    - Striped rows (alternate row colors)
    - Border around cells
    - Clean, readable layout

Validation:

- Check file is CSV format
- Maximum file size: 1MB

Example CSV:

```
Name,Age,City
John,25,Kuala Lumpur
Mary,30,Singapore
David,28,Bangkok
```

Output: HTML table with headers (Name, Age, City) and data rows

Files: XX_C2.php or XX_C2.js

C3: Blog System with Comments (Difficult)

Create a simple blog system with commenting functionality.

Blog Post Management:

- Create new post (title and content)
- View all posts (list view)
- View single post (detail page)
- Delete post

Post Display:

- List page showing:
  - Post title
  - Created date
  - Excerpt (first 100 characters)
  - Link to read full post
- Single post page showing:
  - Full title
  - Full content
  - Created date
  - Comments section

Comment System:

- Add comment form (name, email, comment text)
- Display all comments under the post
- Show comment count on post

Admin Features:

- Simple admin page to:
  - View all posts
  - Create new post
  - Delete posts
  - Delete comments

Validation:

- All fields required
- Validate email format for comments
- Minimum 10 characters for comment text

Database Schema:

posts: id, title, content, created_at

comments: id, post_id, name, email, comment_text, created_at

Files: XX_C3.php or XX_C3.js (+ database schema)

C4: Appointment Booking System (Very Difficult)

Create an appointment booking system.

Requirements:

- Book appointments: select service (dropdown: 4 services), date (date picker), time slot (dropdown: 9am-5pm, hourly slots).
- Customer info: name, email, phone.
- View bookings: list all (date/time/service/customer).
- Check availability: time slot disabled if booked.
- Cancel booking (admin only).
- Today's bookings highlighted.
- Filter bookings: by date, by service.
- Validate: required fields, email format, date not in past, time slot available.
- DB schema: services(id,name,duration_mins,price), bookings(id,service_id,customer_name,customer_email,customer_phone,booking_date,booking_time,status,created_at).
- Files: XX_C4.php or XX_C4.js + schema*

INSTRUCTIONS TO THE COMPETITOR

Please ensure you name the files according to the description and organize your files for assessment.
All assessment is done on the server. No assessment process in workstation.

MARKING SCHEME SUMMARY

SECTION	CRITERION	MEASUREMENT MARKS	TOTAL
A1	General	1.0	1.0
A2	Easy	0.5	0.5
A3	Medium	1.0	1.0
A4	Difficult	1.5	1.5
A5	Very Difficult	2.0	2.0
A6	Easy	0.5	0.5
A7	Medium	1.0	1.0
A8	Difficult	1.5	1.5
A9	Very Difficult	2.0	2.0
A10	Easy	0.5	0.5
A11	Medium	1.0	1.0
A12	Difficult	1.5	1.5
A13	Very Difficult	2.0	2.0
Total		**16.00**	**16.00**